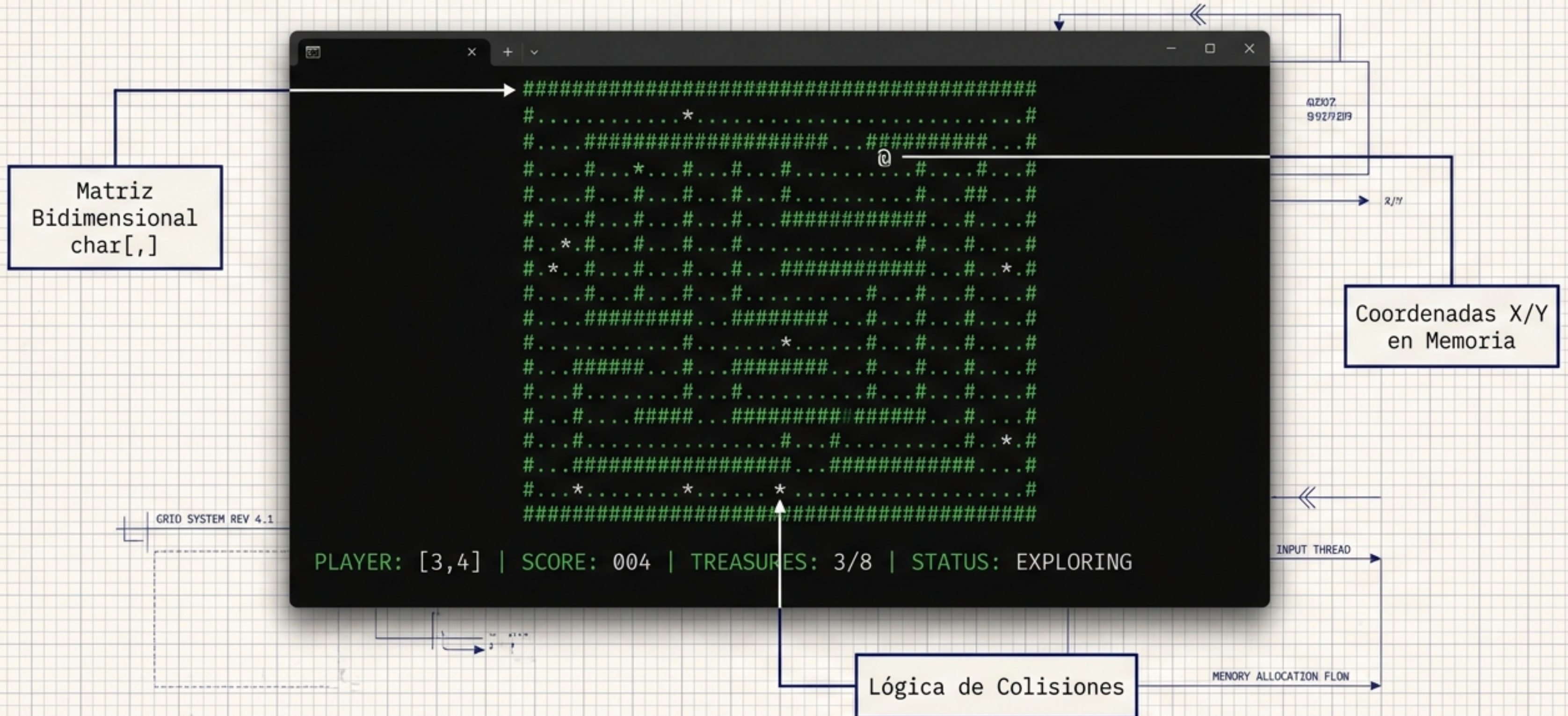


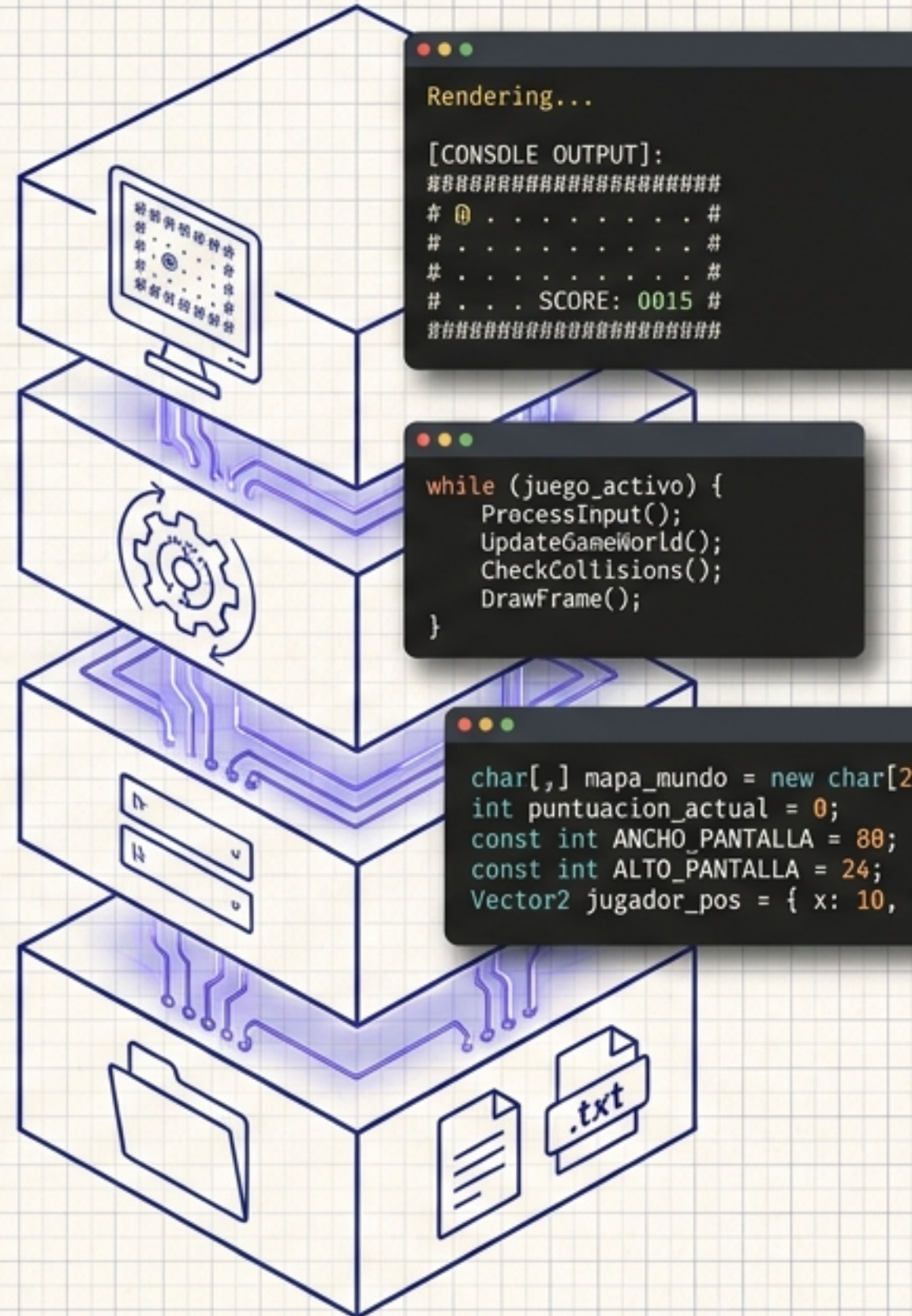
Planos de un Motor C#: El Laberinto de los Tesoros

Arquitectura, persistencia de datos y lógica de estado en un juego de consola.



La Anatomía de un Juego de Consola

- La Vista (Consola)**
Renderizado del mapa interactivo y la interfaz de usuario.
- El Motor Lógico (Bucle Principal)**
Procesamiento de input, colisiones y reglas matemáticas.
- Estado y Datos (Memoria)**
Variables, matrices `char[,]` y constantes que definen el mundo.
- Persistencia (Sistema de Ficheros)**
Lectura/escritura de mapas y rankings en `.txt`.

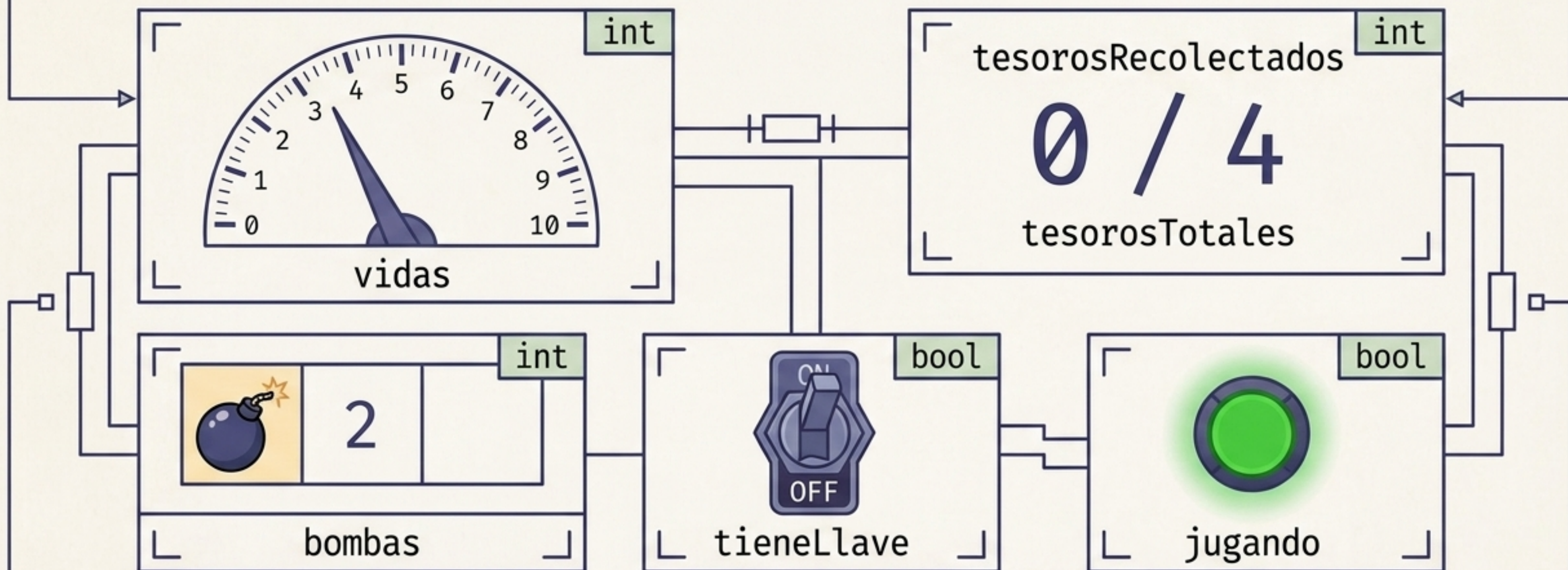


Un juego no es magia; es un estado en memoria que se actualiza rápidamente y se dibuja en pantalla de forma continua.

Matriz de Símbolos: De Texto a Lógica

Símbolo (.txt)	Constante C#	Elemento Visual	Comportamiento Físico
#	PARED	Bloque Sólido	Bloquea el movimiento (IntentarMoverJugador).
.	SUELO_FICHERO	Espacio Vacío	Transitable; se convierte en vacío al cargar.
*	TESORO	Ícono Brillante	Suma 1 a tesorosRecolectados.
T	TRAMPA	Peligro Visible	Resta 1 a la variable vidas.
X	TRAMPA_INVISIBLE	Oculto (Suelo)	Se ve como suelo, pero actúa como trampa.
J	INICIO_JUGADOR	@ (Jugador)	Define jugadorX e jugadorY iniciales.

Dashboard de Estado: Variables en Memoria



El juego completo se controla alterando el valor de estas variables. Cambiar una variable altera la realidad del jugador en el siguiente fotograma.

Persistencia: Preparando el Entorno Fuera de la RAM

RAM - Memoria Volátil

Disco Duro - Almacenamiento Persistente

```
!Directory.Exists("niveles")
```

```
Directory.CreateDirectory()
```



niveles/

nivel1.txt

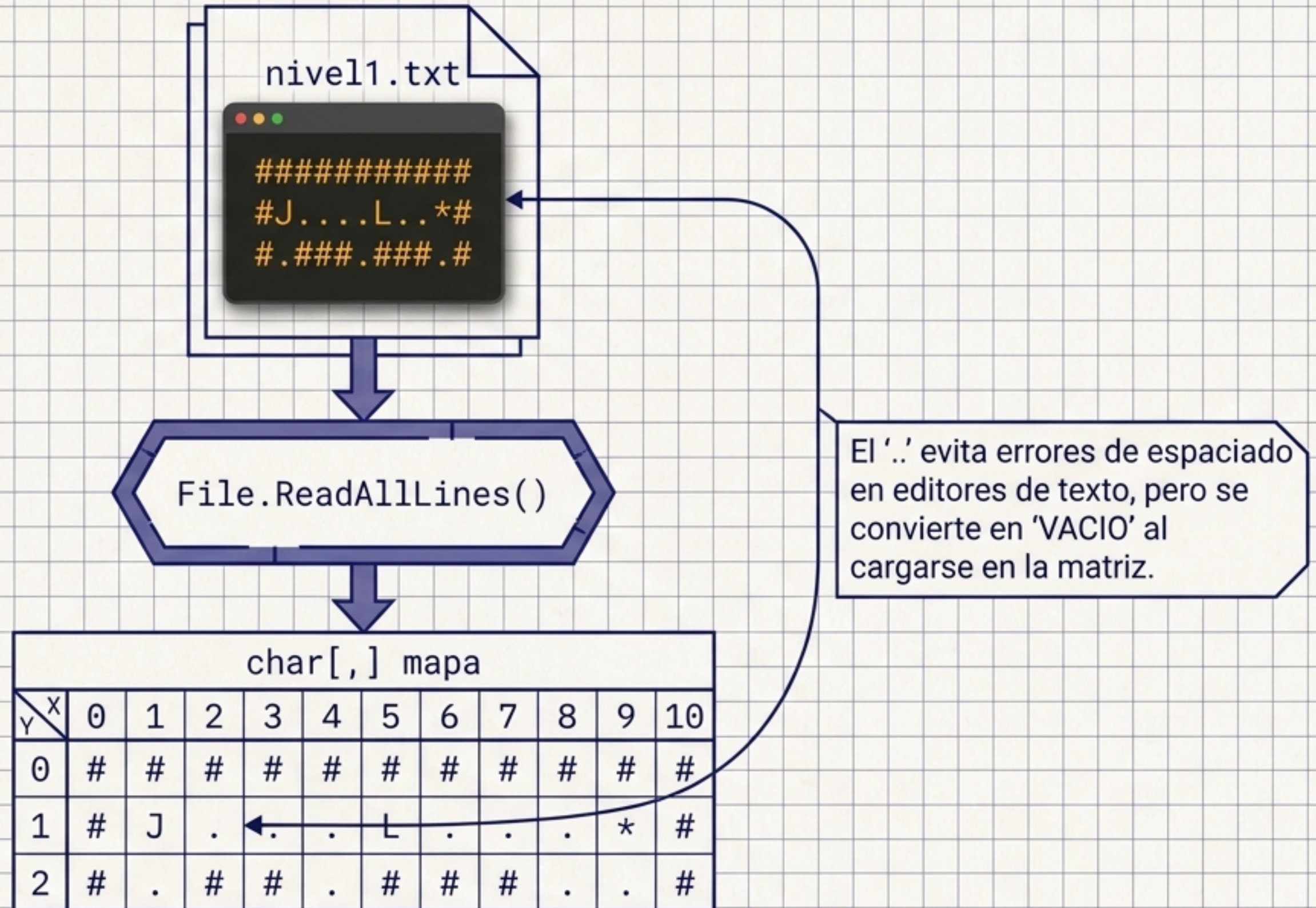
nivel2.txt

nivel3.txt

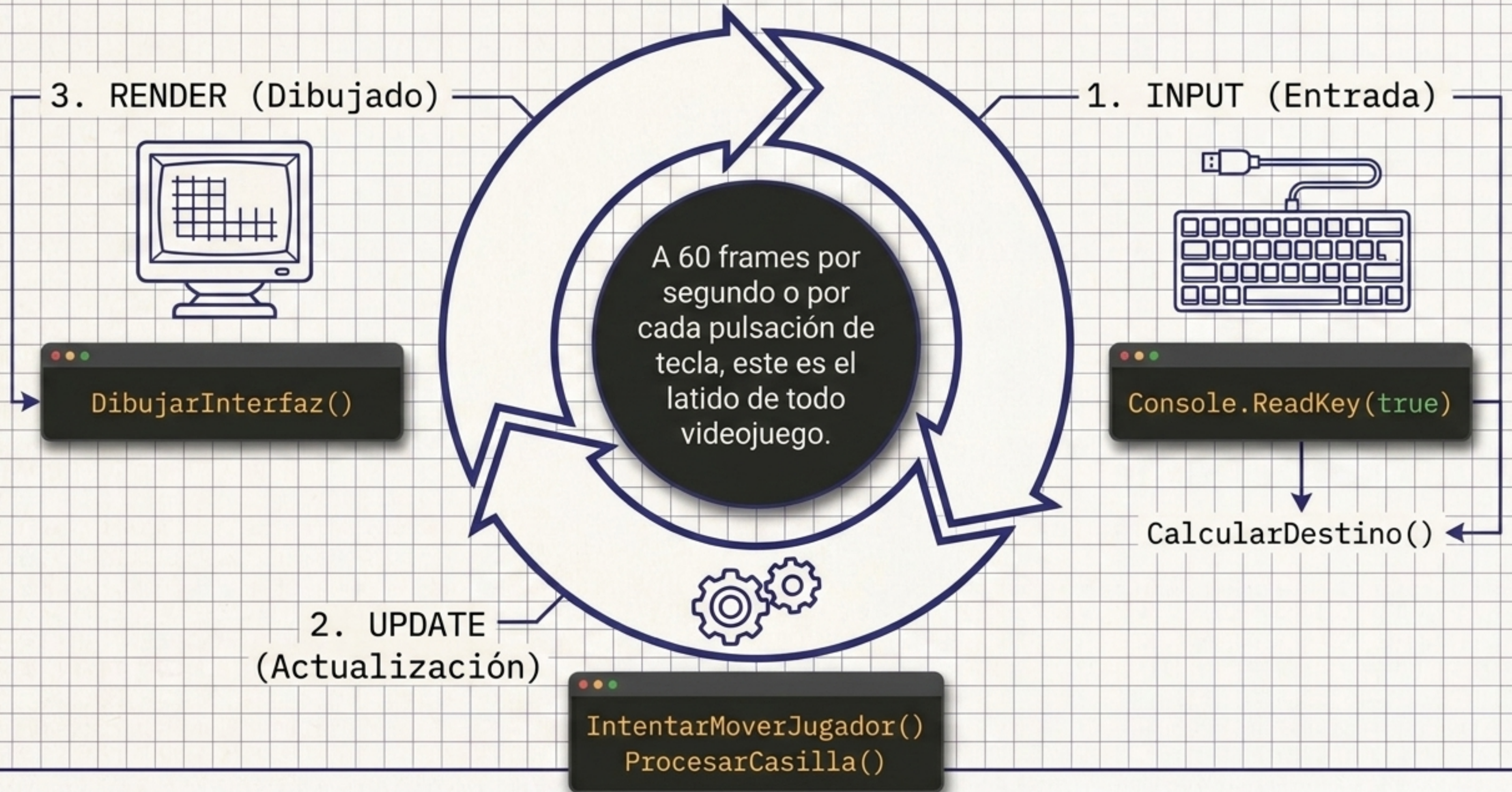
Generados por
CrearNivelSiNoExiste()

Seguridad de Datos: Al usar `!File.Exists`, el código garantiza que si un alumno modifica manualmente un nivel, el programa no sobrescribirá su trabajo al volver a ejecutarse.

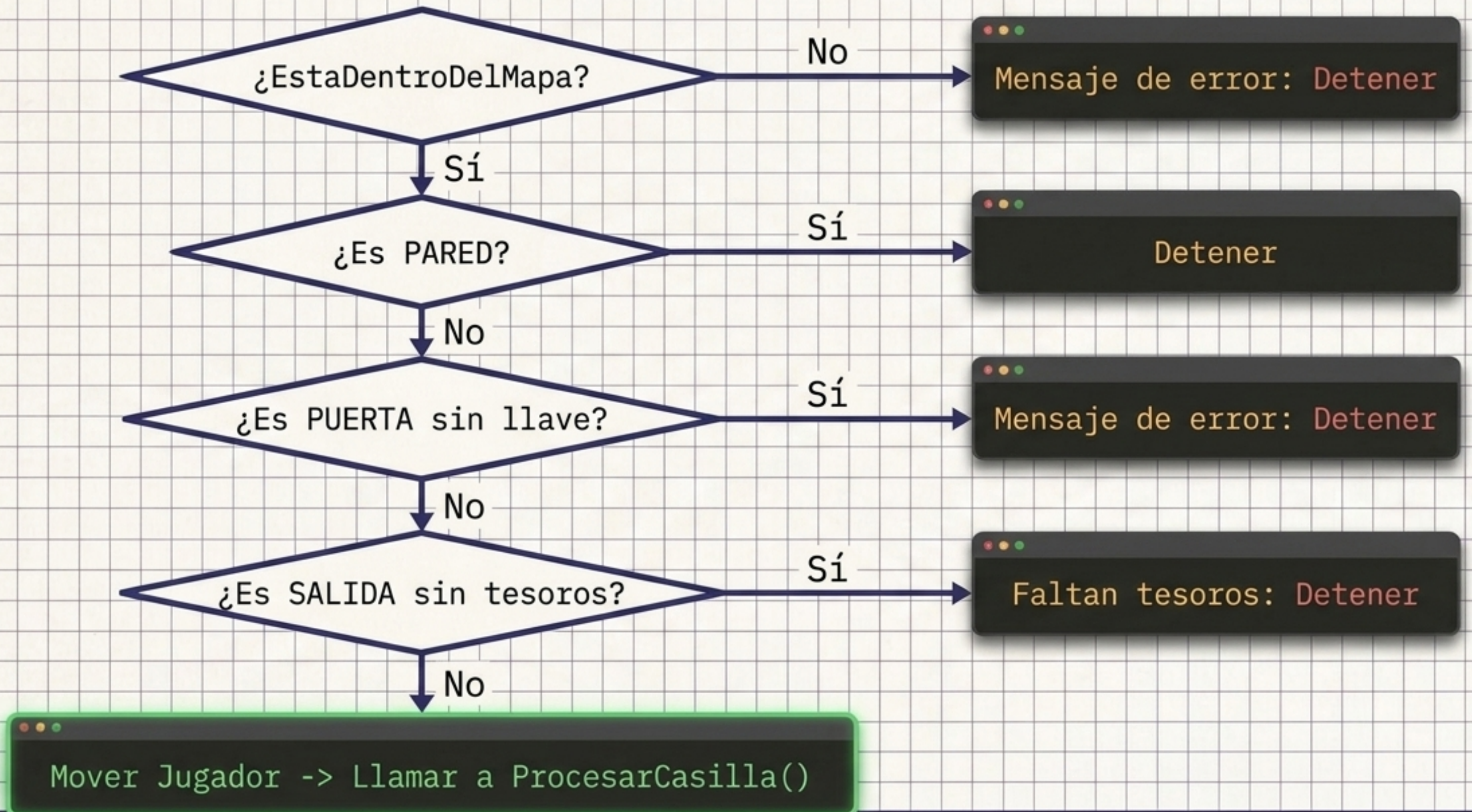
Ingesta de Datos: De Texto Plano a Matriz Bidimensional



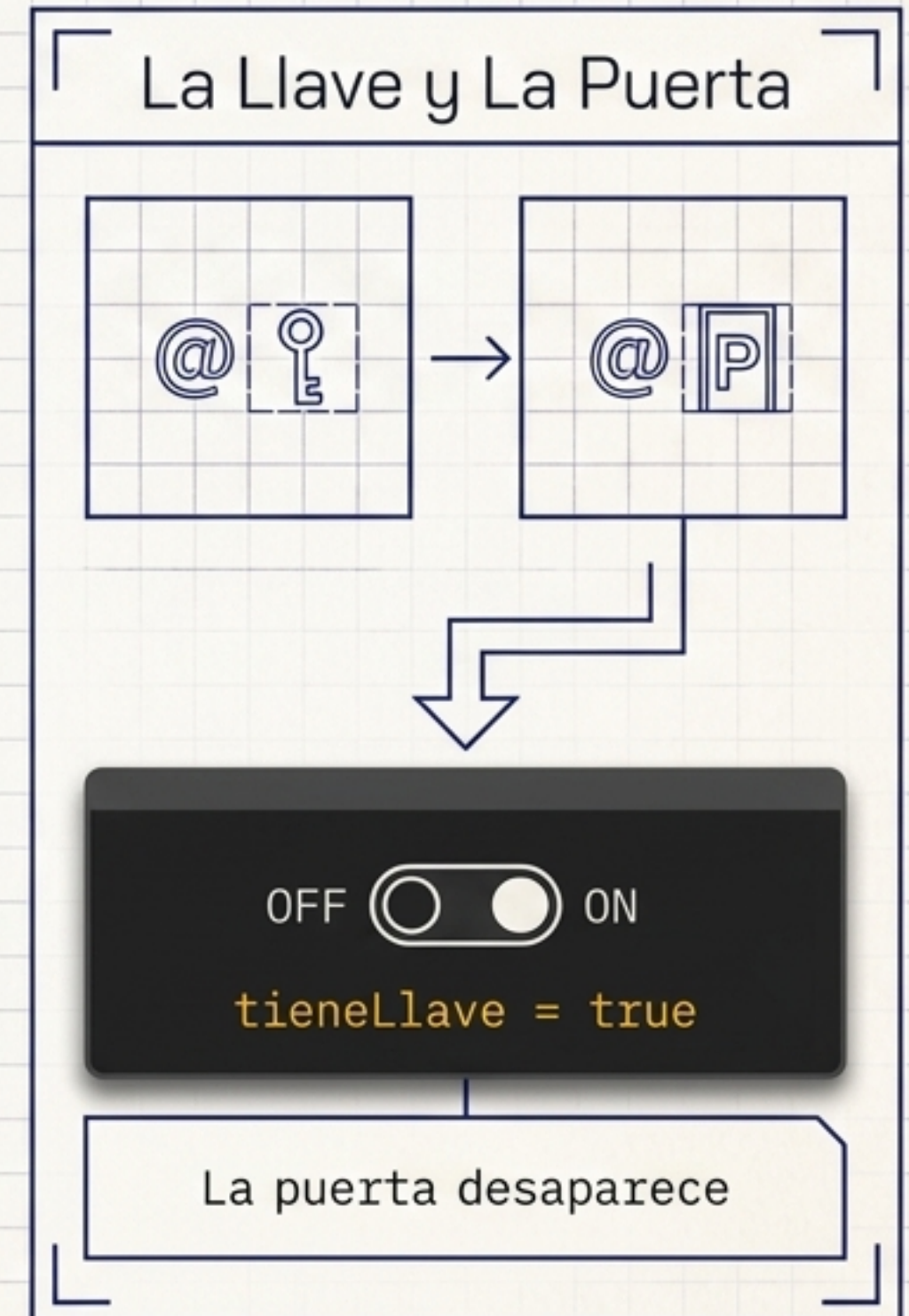
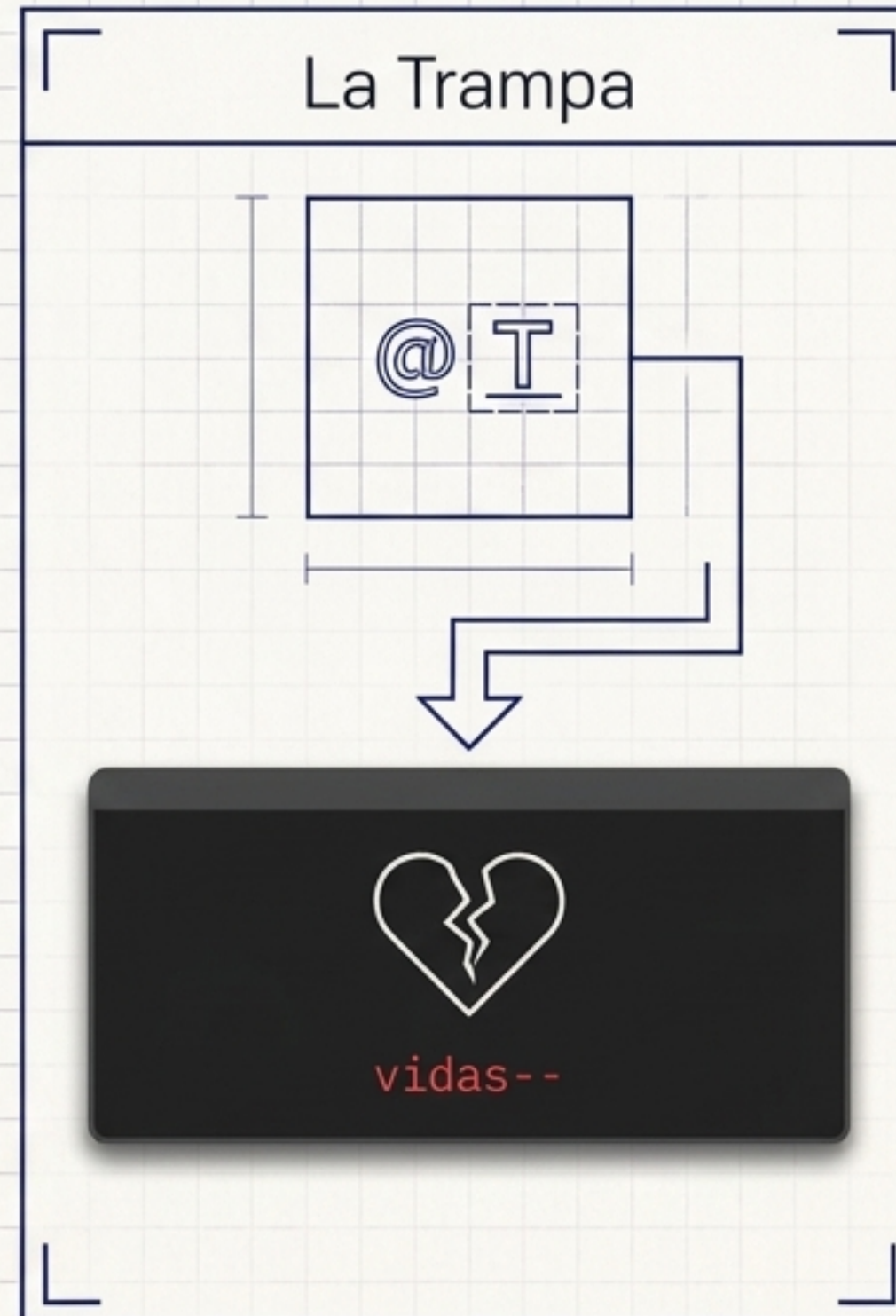
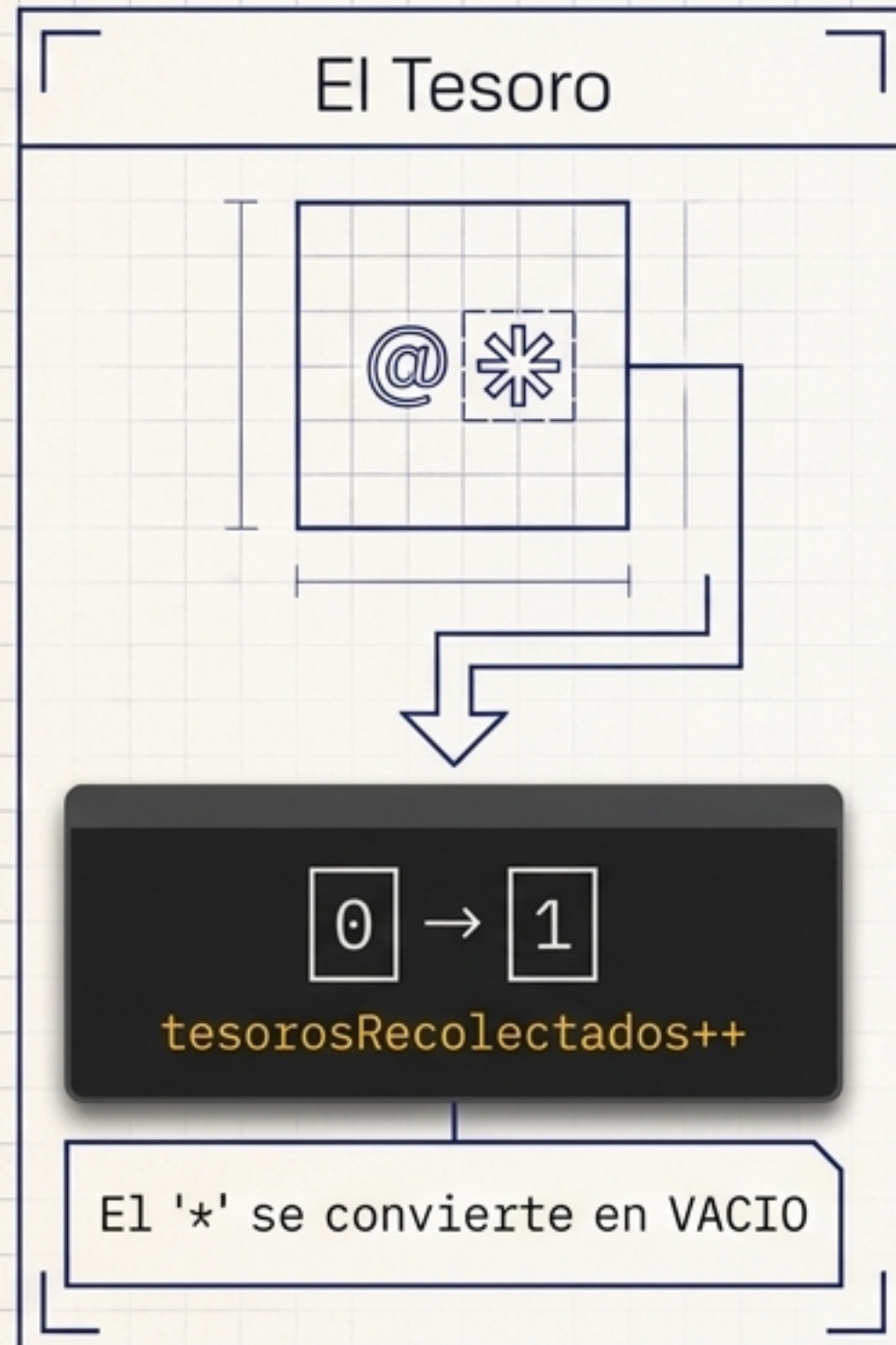
— El Bucle Infinito del Motor (while(jugando)) —



— Lógica de Movimiento: El Árbol de Decisiones —

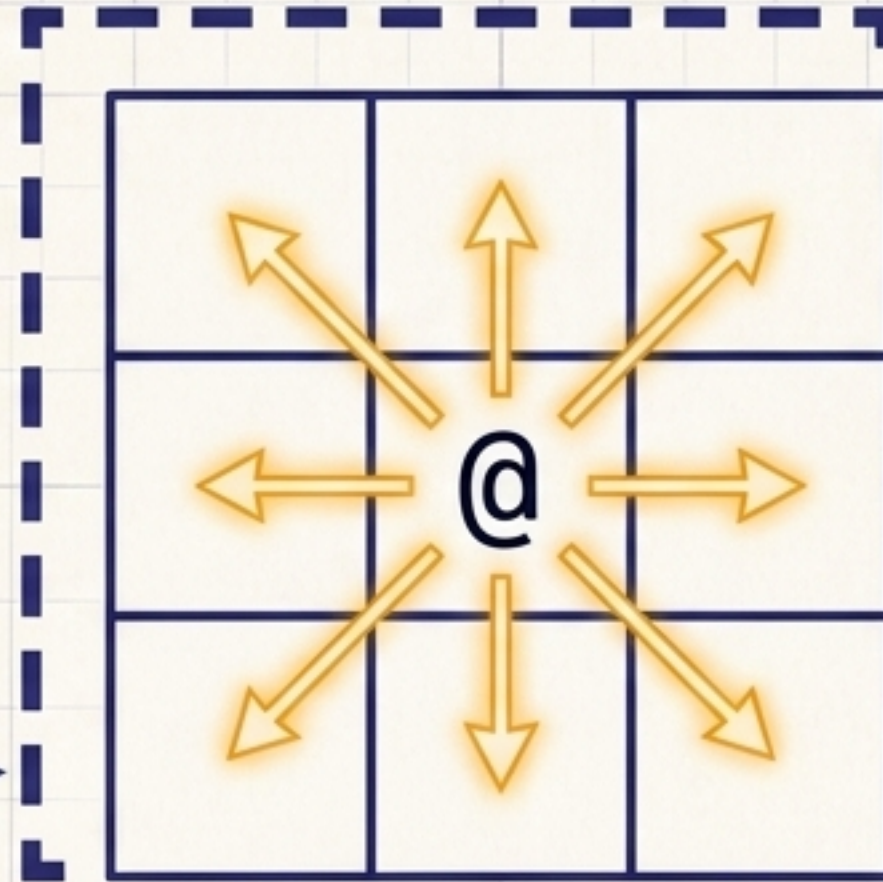


Procesamiento de Casillas: Reacción e Impacto



Manipulación de Matrices: El Radar de la Bomba

Un doble bucle for
recorre desde `jugadorY - 1`
hasta `jugadorY + 1`, y
`jugadorX - 1` hasta
`jugadorX + 1`.



Límites Seguros: La lógica verifica `GetLength(0)` y `GetLength(1)` excluyendo los bordes de la matriz (`y==0`, `x==0`). La bomba nunca destruye el muro exterior, previniendo errores de 'Index Out of Bounds' y fugas del laberinto.

Condiciones de Finalización: Ramificaciones de Salida



Algoritmo de Puntuación: El Balance del Riesgo

Bonificaciones

+ 100 pts × tesorosTotalesPartida
(Objetivo principal)

+ 50 pts × vidas
(Supervivencia)

+ 25 pts × bombas
(Eficiencia de recursos)

Penalizaciones

- 1 pt × segundosTotales
(Velocidad)

- 1 pt × movimientos
(Precisión)

```
if (puntuacion < 0) { puntuacion = 0; }
```

Prevención de deuda: Ningún héroe sale del laberinto debiendo puntos.

Síntesis: El Flujo de Vida del Dato (La Puntuación)

Fase 1 (Top Left): Nacimiento (RAM)

```
1 tesorosTotales = 0;  
2 // Inicialización
```

Se declaran variables base como tesorosTotales=0.

Fase 2 (Mid-Left): Mutación (Bucle)

```
1 while (jugando) {  
2   ...  
3   ProcesarCasilla();  
4   // Detecta colisiones, altera contadores en tiempo real  
5 }
```

ProcesarCasilla() detecta colisiones y altera contadores en tiempo real.

Fase 3 (Center): Cálculo (Matemáticas)

```
1 puntuacionFinal = (tesoros * 100) + (vidas * 50) - (tiempo * 1);  
2 // Aplicar fórmula al salir del bucle
```

Se aplica la fórmula final al salir del while(jugando).

Fase 4 (Mid-Right): Transformación (String)

```
1 string registro =  
  S"{nombreJugador};{puntuacionFinal};{tiempoTotal}";  
2 // Interpolación
```

Interpolación crea cadena: Nombre;Puntuacion;Tiempo.

Fase 5 (Bottom Right): Inmortalidad (Disco Duro)

```
1 File.AppendAllText("ranking.txt", registro + Environment.NewLine);  
2 // Grabar en disco
```

File.AppendAllText('ranking.txt') graba físicamente la cadena.

El software es simplemente el arte de guiar información desde la pulsación de una tecla hasta el disco duro, transformándola en el camino.